

Lab S5 – Mexi Banco con Docker Compose

Paso 1 y 2 – Clonación y Levantamiento del Entorno

Tras clonar el repositorio dentro de mi carpeta de SistemasDistribuidos, levanté los servicios usando docker compose ya dentro de la carpeta clonada. Al esperar un rato para que se levantara tanto la app como la bd, comprobé el estado healthy usando docker compose ps.

```
=> => exporting attestation manifest sha256:86f23631d46fe9e65edf266f91cb9c6940ffbd6039ba0ae2 0.0s
=> => exporting manifest list sha256:dcd867369fe1c3696e614cb1b7eb8f33598ac97059f7ee1d3b458bc 0.0s
=> => naming to docker.io/library/mexi-banco-app:latest 0.0s
=> => unpacking to docker.io/library/mexi-banco-app:latest 0.4s
=> => resolving provenance for metadata file 0.0s
[+] up 5/5
✓ Image mexi-banco-app Built 144.8s
✓ Network mexibanco Created 0.0s
✓ Volume mexi-banco_mexibanco_datos Created 0.0s
✓ Container mexi-banco-db-1 Healthy 5.9s
✓ Container mexi-banco-app-1 Started 6.0s
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	P
ORTS						
mexi-banco-app-1	mexi-banco-app	"java -jar app.jar"	app	6 minutes ago	Up 6 minutes (healthy)	0
.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp						
mexi-banco-db-1	postgres:17-alpine	"docker-entrypoint.s..."	db	6 minutes ago	Up 6 minutes (healthy)	5

```
432/tcp
```

Paso 3 y 4 – Prueba de Endpoints con y sin Volumen

- Arriba y respondiendo

Mexi Banco v05.1 > 1 - Arriba y respondiendo > Salud de la app

GET {{baseUrl}} /actuator/health

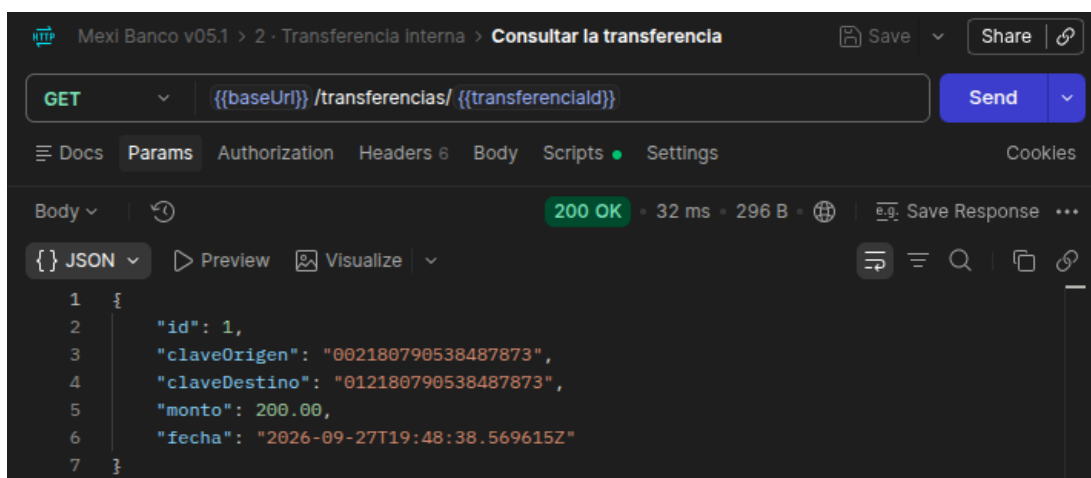
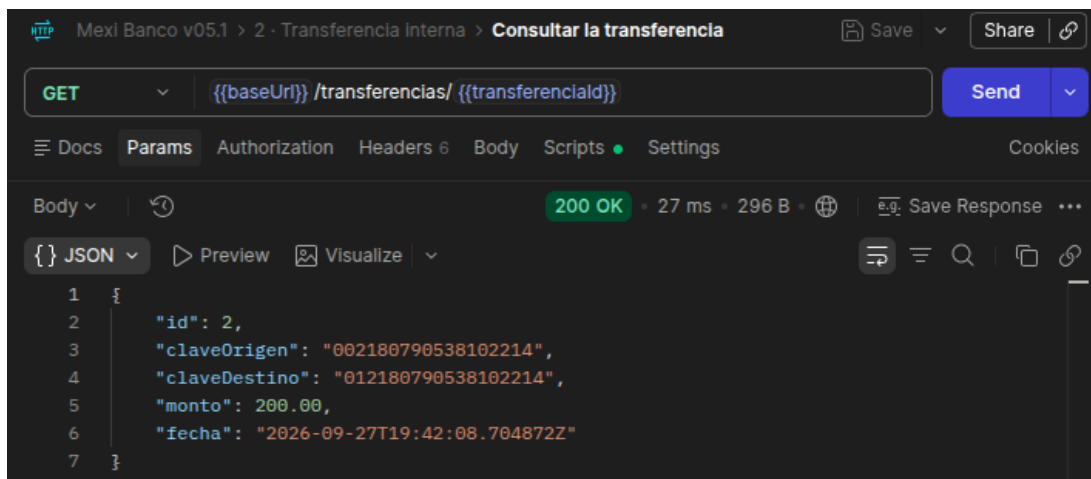
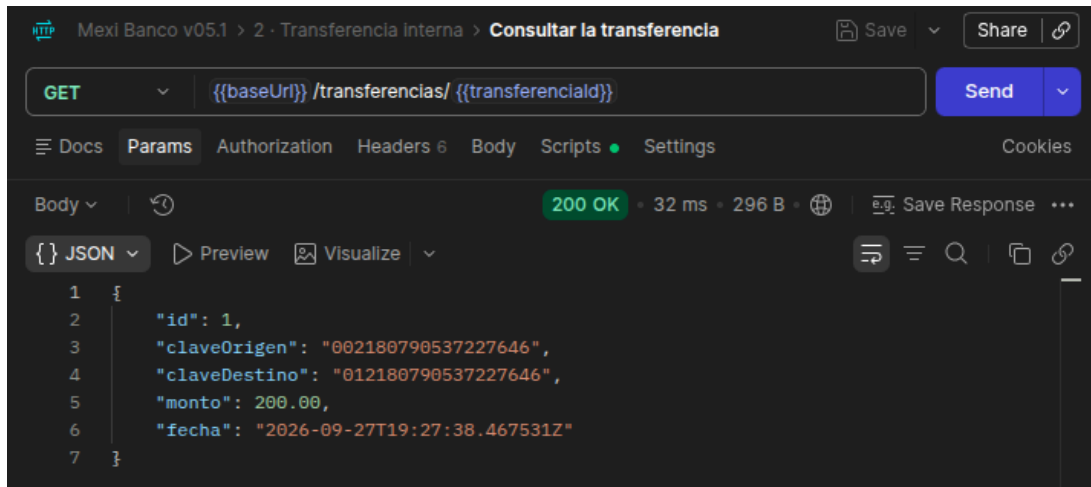
Send

Body 200 OK • 8 ms • 241 B • Save Response

```
{
  "status": "UP",
  "groups": [
    "liveness",
    "readiness"
  ]
}
```

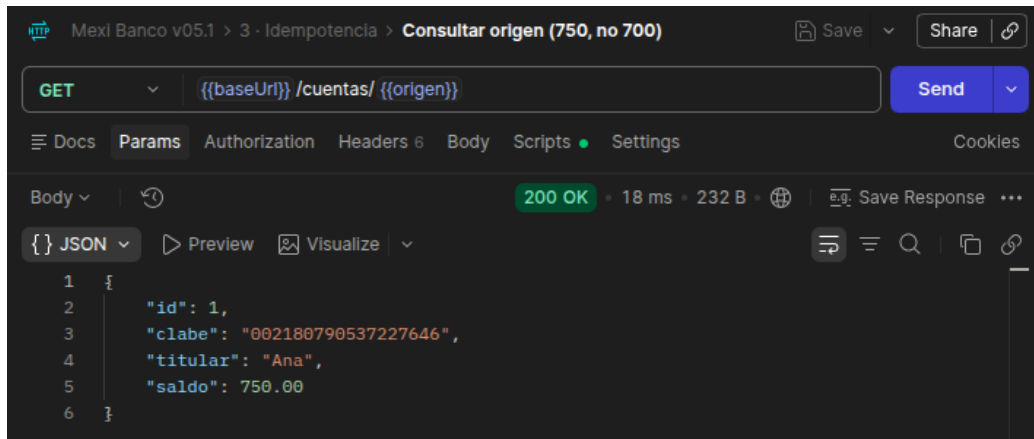
En esta parte solo se verificaba la salud de la app y a su vez se creaba la cuenta bancaria. Los números de la cuenta son aleatorios cada vez que se ejecutan, ya que en la primera ejecución mostró uno, luego otro y al final no volvió al primero que se obtuvo.

- Transferencia interna



Aquí se puede observar lo que pasó en “Arriba y respondiendo”, los números de la cuenta bancaria son diferentes aunque se trate de la misma persona.

- Idempotencia



Mexi Banco v05.1 > 3 · Idempotencia > Consultar origen (750, no 700)

GET {{baseUrl}} /cuentas/ {{origen}}

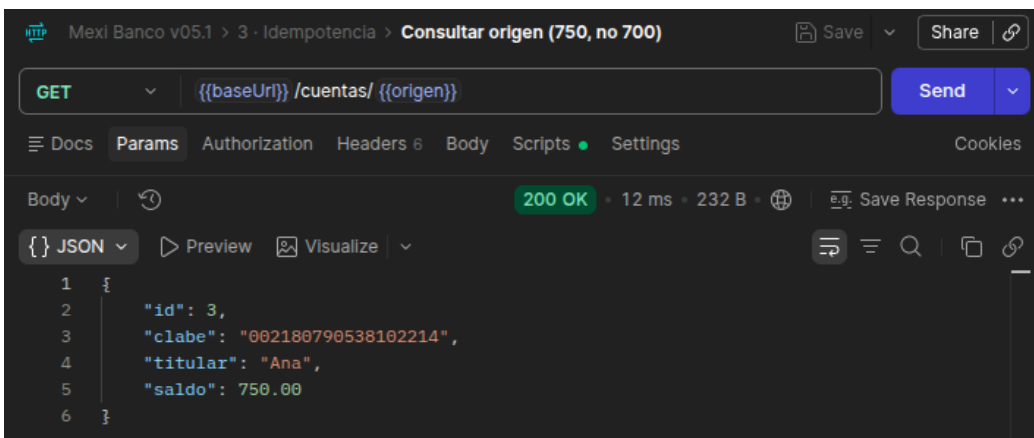
Send

Body

200 OK · 18 ms · 232 B

Save Response

```
1 {
2   "id": 1,
3   "clabe": "002180790537227646",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```



Mexi Banco v05.1 > 3 · Idempotencia > Consultar origen (750, no 700)

GET {{baseUrl}} /cuentas/ {{origen}}

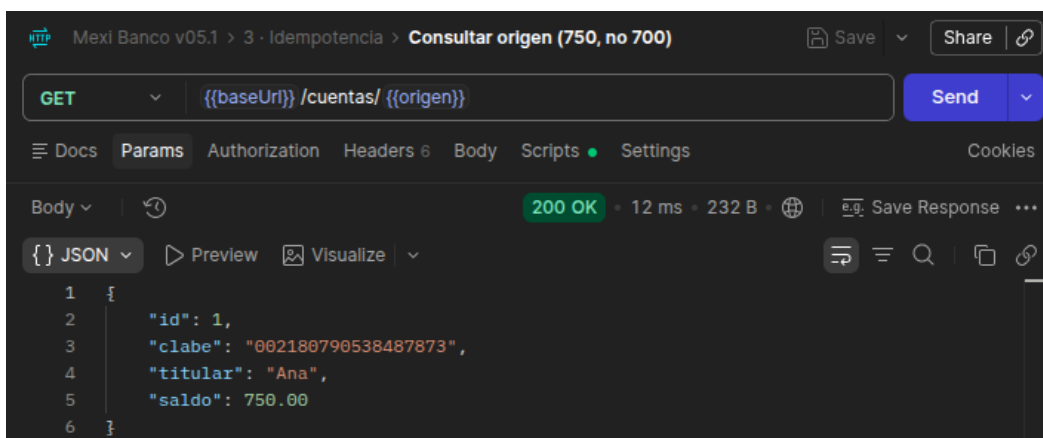
Send

Body

200 OK · 12 ms · 232 B

Save Response

```
1 {
2   "id": 3,
3   "clabe": "002180790538102214",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```



Mexi Banco v05.1 > 3 · Idempotencia > Consultar origen (750, no 700)

GET {{baseUrl}} /cuentas/ {{origen}}

Send

Body

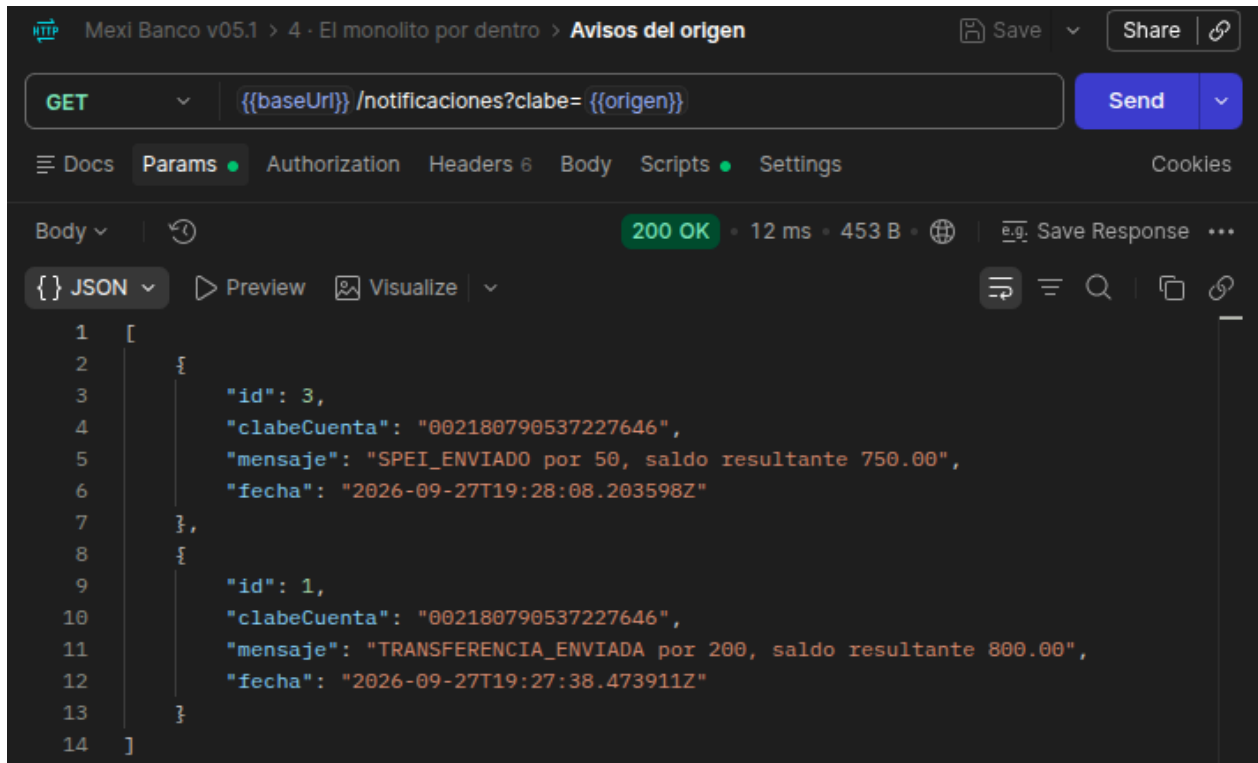
200 OK · 12 ms · 232 B

Save Response

```
1 {
2   "id": 1,
3   "clabe": "002180790538487873",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```

Aunque se trate de cobrar varias veces, gracias a la idempotency-key solo se hará el cobro una vez. Esto para evitar que haya cargos extras y que haya integridad en la información.

- El monolito por dentro



Mexi Banco v05.1 > 4 - El monolito por dentro > Avisos del origen

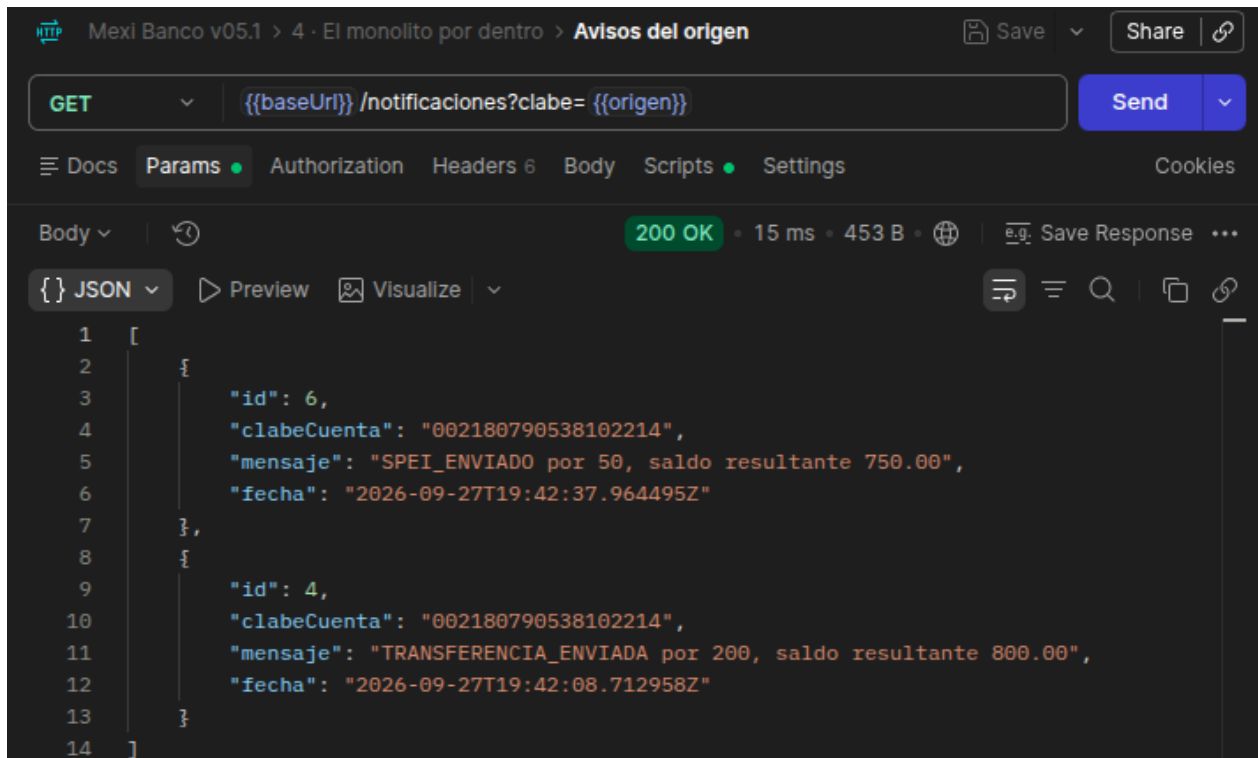
GET `{{baseUrl}} /notificaciones?clabe= {{origen}}` Send

Params Authorization Headers 6 Body Scripts Settings Cookies

Body 200 OK • 12 ms • 453 B Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 3,
4     "clabeCuenta": "002180790537227646",
5     "mensaje": "SPEI_ENVIADO por 50, saldo resultante 750.00",
6     "fecha": "2026-09-27T19:28:08.203598Z"
7   },
8   {
9     "id": 1,
10    "clabeCuenta": "002180790537227646",
11    "mensaje": "TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00",
12    "fecha": "2026-09-27T19:27:38.473911Z"
13  }
14 ]
```



Mexi Banco v05.1 > 4 - El monolito por dentro > Avisos del origen

GET `{{baseUrl}} /notificaciones?clabe= {{origen}}` Send

Params Authorization Headers 6 Body Scripts Settings Cookies

Body 200 OK • 15 ms • 453 B Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 6,
4     "clabeCuenta": "002180790538102214",
5     "mensaje": "SPEI_ENVIADO por 50, saldo resultante 750.00",
6     "fecha": "2026-09-27T19:42:37.964495Z"
7   },
8   {
9     "id": 4,
10    "clabeCuenta": "002180790538102214",
11    "mensaje": "TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00",
12    "fecha": "2026-09-27T19:42:08.712958Z"
13  }
14 ]
```

Mexi Banco v05.1 > 4 · El monolito por dentro > Avisos del origen

GET `{{baseUrl}} /notificaciones?clabe= {{origen}}` Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Body 200 OK · 18 ms · 453 B · Save Response

JSON Preview Visualize

```

1  [
2    {
3      "id": 3,
4      "clabeCuenta": "002180790538487873",
5      "mensaje": "SPEI_ENVIADO por 50, saldo resultante 750.00",
6      "fecha": "2026-09-27T19:49:54.464354Z"
7    },
8    {
9      "id": 1,
10     "clabeCuenta": "002180790538487873",
11     "mensaje": "TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00",
12     "fecha": "2026-09-27T19:48:38.575508Z"
13   }
14 ]

```

Aquí se listan las notificaciones, antes de borrar el volumen se tiene varios mensajes ya que se ejecutaron todos los procesos 2 veces; mientras que al borrar el volumen se tiene la bd completamente vacía.

- Cuando algo sale mal

Mexi Banco v05.1 > 5 · Cuando algo sale mal > CLABE que no existe (404)

GET `{{baseUrl}} /cuentas/999999999999999999` Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Body 404 Not Found · 44 ms · 317 B · Save Response

Find a valid CLABE Diagnose Decline

JSON Preview Debug with AI

```

1  {
2    "type": "about:blank",
3    "title": "Not Found",
4    "status": 404,
5    "detail": "No existe la CLABE 999999999999999999",
6    "instance": "/cuentas/999999999999999999"
7  }

```

Aquí no hubo ningún problema, solo se mostraron algunos tipos de errores, pero que no afectaron en nada a los datos.

Paso 5 y 6 – Revisión de Código y Configuración

Al momento de abrir el proyecto en VSCode, en la carpeta “src/main/java/mx/mexibanco”, pude observar la estructura del monolito. También pude identificar los cinco módulos principales y confirmar que cada uno de ellos está organizado en cuatro capas: controlador, servicio, repositorio y entidad.

Al analizar el archivo de configuración de Compose, vi cómo se conectan los servicios mediante la red, la declaración del volumen y la configuración del healthcheck. En el Dockerfile, observé el esquema multi-capa, separando la fase de compilación de la de ejecución de la aplicación.